

ON-DEVICE AGENTIC AI · IOS

Anu

Architecture & Engineering Documentation

A voice-first, multimodal iOS assistant whose brain — **Gemma 4 E4B** — runs entirely on the device via Google's LiteRT engine. An on-device ML router decides, per message, what kind of task it is and which engine should answer it. Private by default; the cloud is a sanitized, optional, user-gated fallback.

Author: Sai Sandeep Kumar Vijayarao

Repository: github.com/sandeepvijayarao09/Anu-iOS

Scope: ~11.6K lines of Swift 6 · 5 Xcode targets · 130+ tests · iOS 17–26.5

Document: Complete system architecture & subsystem reference

Contents

1. The big picture
2. Design principles
3. Targets & build
4. Source layout
5. The Agent layer (the core)
6. The Models layer (the swappable brain)
7. Tools & tool use
8. Connectors — external & remote tools
9. Cloud API clients
10. App, Intents & Workflows
11. UI
12. Utilities (cross-cutting)
13. End-to-end: one message, start to finish
14. Testing
15. Key invariants (quick reference)
16. The agentic system, in depth (with code)
17. On-device execution, in depth (with code)

1. The big picture

Anu is built around a single idea: **most requests should be answered on the device, and the user should always be able to see exactly what (if anything) left it.**

Everything is coordinated by one `@MainActor` singleton, `AgentOrchestrator`, the single source of truth for both the SwiftUI UI and the App Intents (Siri / Shortcuts / Action Button / Widget). A user message flows through three on-device decision stages and then one of several execution modes:

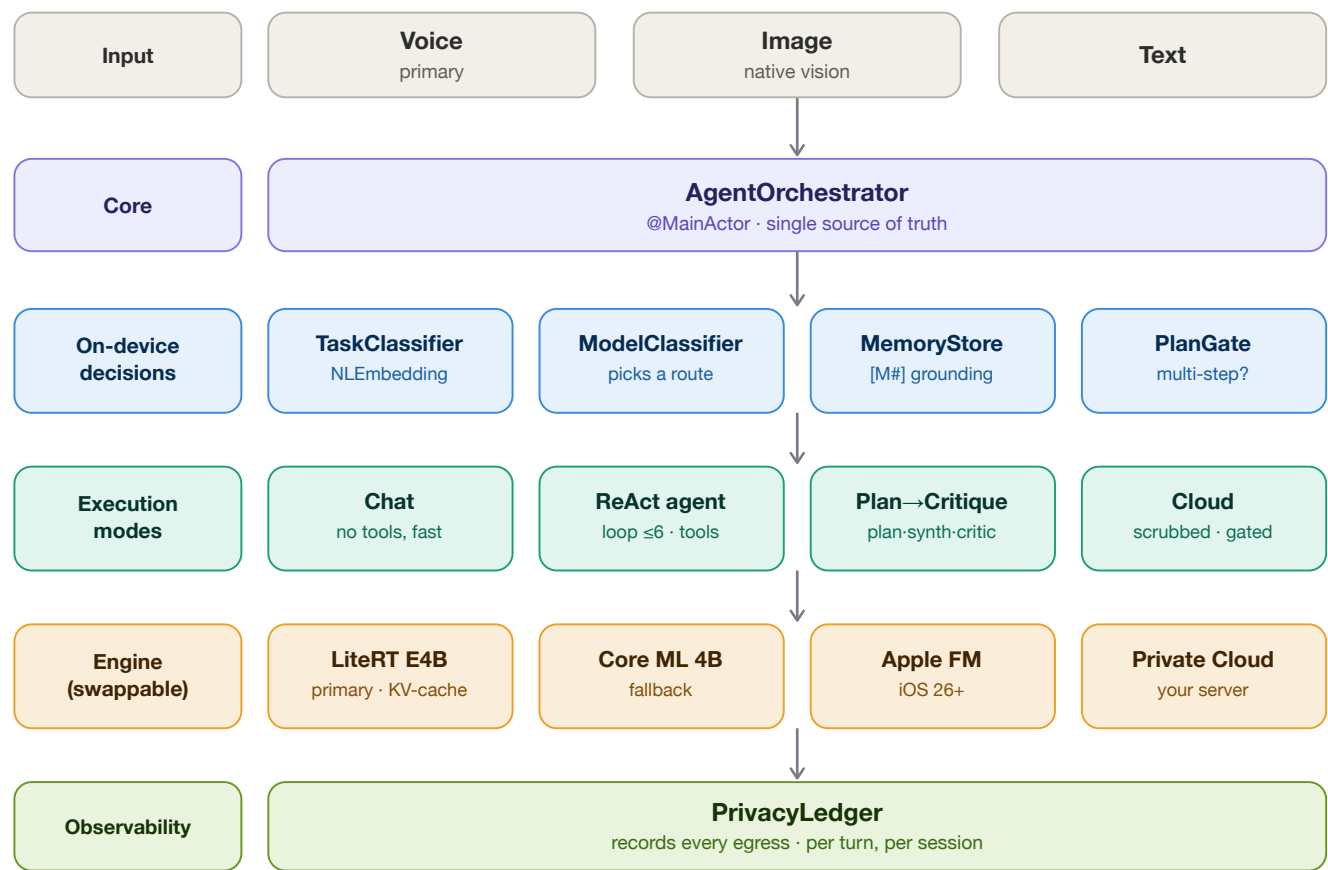


Figure 1 — Layered architecture: input → orchestrator → on-device routing → execution modes → swappable engine, with the privacy ledger observing all egress.

2. Design principles

Principle	How it shows up in the code
On-device first	<code>ModelClassifier</code> keeps everything local unless a task clearly benefits from cloud <i>and</i> a key/endpoint is configured.
Single source of truth	<code>AgentOrchestrator.shared</code> is the only place conversation state lives; UI and Siri both drive it.
Glass-box reasoning	Every step is recorded as a <code>ReasoningStep</code> and rendered in <code>AgentTraceView</code> .
Privacy is observable	<code>PrivacyLedger</code> records every egress; a per-turn chip shows "everything on-device" or what left.
No silent fakes in production	A missing model becomes <code>UnavailableModel</code> (an explicit error); the <code>ScriptedModel</code> test double is <code>#if DEBUG</code> only.
Swappable brain	All engines conform to one <code>LocalLanguageModel</code> protocol; <code>ModelFactory</code> selects at launch.
Bounded everything	ReAct ≤ 6 iterations, planner ≤ 6 steps, one critic pass, one revision, 30s tool timeouts.

3. Targets & build

The app ships as **five Xcode targets**, all generated by code:

Target	Type	Purpose
<code>Anu</code>	iOS app	The main assistant.
<code>AnuTests</code>	unit tests	~51 test files (tool parsing, classifiers, tokenizer, PII, memory, orchestrator...).
<code>AnuUITests</code>	UI tests	Drive the real chat flow against the DEBUG <code>ScriptedModel</code> .
<code>AnuWidget</code>	Widget extension	Surfaces saved Workflows; runs them via <code>RunWorkflowIntent</code> (deliberately model-free).
<code>AnuShare</code>	Share extension	Accepts shared content and enqueues it for the app.

- `generate_xcodeproj.py` is the **source of truth** for the project (project-as-code, like XcodeGen/Tuist). The `.xcodeproj` / `.xcworkspace` are build artifacts. It bundles a present `.litertlm` / `.mlpackage` automatically and omits them when absent.
- **CocoaPods** provides MediaPipe (LiteRT runtime) — hence you open `Anu.xcworkspace`, not the `.xcodeproj`.
- **App Group** `group.com.anu.app` is shared by the app and its extensions for Workflows and the inbox queue.
- Swift 6 structured concurrency throughout: actor-isolated engines, a `@MainActor` orchestrator, `AsyncStream` generation wired to per-backend cancellation.

4. Source layout

Sources/Anu/	
├─ App/	App entry, root view, Siri/Shortcuts & Writing-Tools intents
├─ Agent/	The agentic core: orchestrator, routing, planning, critique, memory, sessions
├─ Models/	Engine backends behind one protocol + the selection factory
├─ Tools/	Tool protocol + registry + built-in tools
├─ Connectors/	External tools: MCP servers, REST endpoints, app launching
├─ API/	Cloud clients: Gemini + Private Compute (+ request/response models)
├─ Workflows/	Saved prompt recipes shared with the widget/share extensions
├─ UI/	SwiftUI views (chat, trace, memory, settings, connectors, privacy...)
└─ Utilities/	Keychain, PII scrub, persistence, speech I/O, streaming, attestation

5. The Agent layer (the core)

This is what makes Anu *agentic* rather than a chat wrapper. It lives in `Sources/Anu/Agent/`.

5.1 AgentOrchestrator — the hub

`AgentOrchestrator` is a `@MainActor final class` exposed as `.shared`. It owns the published UI state (`messages`, `status`, `reasoningSteps`, `isThinking`, `pendingConfirmation`), the swappable `model`, the `ToolRegistry`, conversation history, and the multi-session store. Its responsibilities are split:

- **AgentOrchestrator.swift** — init, tool registration, the `run(userMessage:imageData:)` entry point, classification + routing, tool execution (30s timeout), and the confirmation gate.
- **AgentOrchestrator+Modes.swift** — the three generation modes: `chatTurn()`, `reactLoop()`, `cloudTurn()`.
- **AgentOrchestrator+Sessions.swift** — privacy-sandbox lifecycle (create / switch / rename / delete / persist / ephemeral sweep).
- **PlanPipeline.swift** (extension) — the multi-step planner→executor→critic pipeline.

`maxIterations = 6` bounds every ReAct loop. A `completeHeadless(system:user:)` path powers Siri inline answers and Writing Tools with no UI side effects.

5.2 Routing — three on-device decisions

Component	Role
TaskClassifier	Embeds the message with Apple's <code>NLEmbedding</code> and picks the nearest task centroid (<code>casualChat</code> , <code>math</code> , <code>webInfo</code> , <code>codeGen</code> , <code>longWriting</code> , <code>generalQA</code>), with a keyword-rule fallback. Milliseconds, no network.
ModelClassifier	Maps <code>(task, confidence, cloudAvailable)</code> → a route: <code>.onDeviceChat</code> , <code>.onDeviceAgent</code> , or <code>.cloudEscalate</code> . Confidence floor 0.45; low confidence → agent path. Force-routes to the agent when a device tool (time/calendar/reminders/contacts) is needed.
MessageRouter	Legacy heuristic chat-vs-agent router, retained for reference; superseded by the two classifiers.
RoutingDecision / ResponseParser	Parses raw model output into <code>answerLocally</code> vs <code>callTool(...)</code> vs <code>escalateToGemini</code> , extracting the first balanced <code>{"tool_call": ...}</code> JSON (tolerant of code fences and truncation).

The route is then overridden in two cases: an attached image or "fast mode" forces `.onDeviceChat` ; a needed device tool forces `.onDeviceAgent` .

5.3 Three execution modes

1. **Chat** (`chatTurn`) — single-shot, no tools, warm persona, lowest latency.
2. **ReAct agent** (`reactLoop`) — the bounded reason→act→observe loop (≤ 6 iterations). Each turn: build an agent prompt with the tool catalog, stream the model, parse a `RoutingDecision` , then finish, run a tool, or escalate. Tool JSON is hidden from chat and surfaced as a tool card + trace step.
3. **Cloud** (`cloudTurn`) — resolves the provider, **sanitizes PII**, records the egress, runs the escalation tool, folds the result back into history.

5.4 Plan → Execute → Critique pipeline

For requests that are *clearly* multi-step, `PlanGate.shouldPlan()` (length ≥ 24 , explicit sequencing, lists, or ≥ 2 distinct task domains) opts the message into the heavier pipeline — *only* when the user has also enabled "deep reasoning":

1. **Plan** — `Planner.makePlan()` decomposes the goal into ≤ 6 ordered steps, each assigned to a **specialist** (`researcher` , `coder` , `mathematician` , `writer` , `generalist`). One deterministic call; any failure degrades to a single step.
2. **Execute** — each step runs a **headless ReAct loop** (`runReActHeadless`) with the specialist's focused prompt and a *restricted tool subset*. Results accumulate in a `Scratchpad` ; a shared iteration budget is decremented across steps.
3. **Synthesize** — `synthesize()` combines the scratchpad into one answer (single-step plans skip this).
4. **Critique** — `Critic.review()` returns `.approved` or `.revise(instruction)` ; one revision pass max. Ambiguity defaults to approved.
5. **Present** — the final text streams into the chat bubble.

5.5 State, memory & sessions

Component	Role
AgentMessage	Message model (user/assistant/toolCall/toolResult/system) with optional <code>imageData</code> for vision.
AgentState	<code>AgentStatus</code> (drives the status bar) and <code>ReasoningStep</code> / <code>ReasoningStepKind</code> (drive the trace timeline).
MemoryStore	NotebookLM-style editable notes. Every prompt carries a standing profile (newest 3 notes) + query-relevant older notes retrieved by embedding similarity, injected as numbered <code>[M#]</code> grounding and cited in the trace.
ConversationWindow	Trims history to ~2800 chars (~700 tokens) before each prompt, sliding the window to start at a user turn.
ChatSession	A privacy sandbox : independent history, <code>MemoryScope</code> (<code>global</code> / <code>isolated</code>), <code>ephemeral</code> ("incognito") flag, optional per-session model override.
PrivacyLedger	Per-turn, per-session record of every egress (<code>cloud</code> , <code>privateComputeServer</code> , <code>mcpServer</code> , <code>restHost</code> , <code>appLaunch</code>) with payload summary, redaction count, byte count.
CloudProvider	Resolves which cloud backend handles escalation (<code>auto</code> prefers a configured private server, else Gemini).

6. The Models layer (the swappable brain)

All engines conform to one protocol, `LocalLanguageModel` — a `Sendable` protocol whose *implementations* are each an `actor` :

```
protocol LocalLanguageModel: Sendable {
    var modelName: String { get }
    func load() async throws
    func generate(prompt: String, config: GenerationConfig) async throws ->
    AsyncStream<String>
    func generate(prompt: String, image: CGImage?, config: GenerationConfig) async throws
    -> AsyncStream<String>
    func resetSession() async // drop KV cache on sandbox switch
}
```

6.1 Backends

Backend	Notes
LiteRTGemmaModel (primary)	Gemma 4 E4B via MediaPipe <code>LlmInference</code> . KV-cache + session reuse across turns (feeds only the new delta when the prompt extends the cached prefix, via <code>SessionContextTracker</code>). Native vision modality. Uses <code>StreamStopFilter</code> because MediaPipe generates past <code><end_of_turn></code> . The entire MediaPipe surface is confined to this one file.
GemmaModel (Core ML fallback)	Gemma 3 4B Core ML, fixed-shape <code>[1, seqLen]</code> I/O, no KV cache; manual sampling (temperature + top-k/top-p + repetition penalty). iOS 18+ (int4 per-block quant).
FoundationModelBackend	Apple Intelligence on-device model (iOS 26+). Handles Apple's cumulative streaming snapshots by emitting only the new suffix.
PrivateCloudModel	Streams from the user's private compute server via <code>PrivateComputeClient</code> ; selectable as a brain <i>and</i> an escalation target. Rotates session id on sandbox switch.
ScriptedModel (<code>#if DEBUG</code>)	Deterministic test double, reachable only via <code>-scripted_model YES</code> . Recognizes <code>[[PLANNER]]</code> / <code>[[CRITIC]]</code> / <code>[[SYNTH]]</code> sentinels so the whole pipeline is UI-testable.
UnavailableModel	Explicit-error placeholder — no silent fake in production.

6.2 Selection — ModelFactory

`ModelFactory.makeModel()` : (1) `#if DEBUG` and `-scripted_model` set → `ScriptedModel` ; (2) otherwise honor the user's `UserDefaults("selected_model")` choice; (3) `.automatic` (default) → the priority chain:

```
LiteRT (KV-cached) → Core ML (fixed-shape) → UnavailableModel
```

Supporting types: `GenerationConfig` / `ModelConfig` hold sampling presets (`.chat` , `.deterministic` , `.agentFromSettings` — the last clamps temperature ≤ 0.45 for reliable tool-call JSON); `ModelCatalog` is the registry + availability probe; `ModelManager` is the observable selection bound to Settings. Tokenization is protocol-based: `GemmaTokenizer` (HuggingFace `tokenizer.json` , greedy longest-match + byte fallback) with a character-level `FallbackTokenizer` .

7. Tools & tool use

A tool declares a `name`, a `description`, an optional `JSONSchema` of `parameters`, a `sideEffect` (`.readOnly` vs `.external(capability:)`), a `requiresConfirmation` flag, and an async `execute(arguments:) -> String`. The `sideEffect` classification is what the confirmation gate keys off. `ToolRegistry` is a `@MainActor` registry where built-in tools always win name collisions and connector tools are swapped in atomically. Tools are exposed to the model by name + description + schema in the agent system prompt.

Tool	Device API / permission	What it does
<code>CalculatorTool</code>	none	Safe recursive-descent expression evaluator (rejects NaN/Inf).
<code>WebSearchTool</code>	network	Brave Search; graceful "unavailable" without a key.
<code>DateTimeTool</code>	none	Current time + relative date math (grounds "next Friday").
<code>UnitConverterTool</code>	none	Offline conversions across 8 categories via <code>Measurement</code> .
<code>RemindersTool</code>	EventKit (Reminders)	Create / list reminders.
<code>CalendarTool</code>	EventKit (Calendar)	Create / list events.
<code>ContactsTool</code>	Contacts (read-only)	Look up phone/email by name.
<code>ImageGenerationTool</code>	Image Playground (iOS 18.4+)	The agent can "draw"; result is presented in chat.
<code>EscalateToGeminiTool</code>	network → Gemini	PII-sanitizes task + context, records the ledger, calls Gemini.
<code>EscalateToPrivateCloudTool</code>	network → private server	Same PII funnel/ledger; tells the client not to double-record.

Permission-gated tools degrade gracefully (a denied permission returns a message, never a crash).

8. Connectors — external & remote tools

`Sources/Anu/Connectors/` lets the agent use tools that live outside the app, all surfaced as ordinary `Tool`s in the registry.

- **MCP (Model Context Protocol) servers** — the SDK is kept behind `#if canImport(MCP) / @_implementationOnly`. `ConnectorConfig` defines `MCPServerConfig` + `MCPAuth` (`none` / `bearer` / `oauth`), persisted in the **Keychain**. `ConnectorManager` is the bridge between Settings and the registry. `MCPConnection` (an actor) holds one live connection (12s list / 30s call timeouts). `MCP0Auth` bridges the SDK's OAuth 2.1/PKCE flow via `ASWebAuthenticationSession`. `MCPToolProxy` adapts a remote tool into a `Tool` with a namespaced name `mcp__<server>__<tool>`, marks it `.external(.connector)` + `requiresConfirmation`, and logs egress. `MCPCatalog` is a curated one-tap list; `MCPRegistryClient` searches the official MCP registry (HTTPS-only).
- **REST connectors** — `RESTConnectorTool` exposes one user-configured endpoint with a fixed host (the agent supplies path/query/body; writes require confirmation).

- **App launching / Shortcuts** — `AppLauncherTool` opens allowlisted URL schemes (`maps`, `tel`, `mailto`, `shortcuts` ...); always confirmation-gated and ledgered.
- **UCPDDiscovery** — discovers a merchant's MCP endpoint via the Universal Commerce Protocol (`/.well-known/ucp`); read-only.

Lifecycle of an external MCP tool: add server → optional OAuth sign-in (browser, token to Keychain) → `refresh()` connects, lists tools, wraps each in `MCPToolProxy`, and atomically registers them → the model sees them in the prompt → on a tool call the proxy gate-checks capability + confirmation → executes over the live connection → logs egress → returns text to the loop.

9. Cloud API clients

`Sources/Anu/API/` holds two structurally-parallel HTTPS clients; both stream via a shared `SSEParser`.

- **GeminiClient** — Google Gemini (`gemini-2.0-flash`), authed by a Keychain-stored API key, `generate()` + streaming `stream()`, typed errors.
- **PrivateComputeClient** — the operator-managed private server. Authed by **App Attest** headers (not a URL key), HTTPS enforced, per-sandbox ephemeral `sessionId`, `preflight()` reachability check.

10. App, Intents & Workflows

- **AnuApp** — `@main`; drains the inbox queue on activation and sweeps ephemeral sandboxes on backgrounding.
- **ContentView** — root `NavigationStack` hosting `ChatView` + toolbar to sessions/settings.
- **App Intents** — `AskAnuIntent` (opens app, pipes into the loop), `AskAnuInlineIntent` (headless, spoken), `StartVoiceIntent` ("Talk to Anu" / Action Button, auto-opens the mic), and `AnuShortcuts`. **Writing Tools** intents (Summarize/Rewrite/Proofread) run through `completeHeadless`.
- **Workflows** — saved prompt recipes (`Workflow`) managed by `WorkflowManager`, stored in the App Group (`WorkflowStore`) so the widget/share extensions see them, indexed into Spotlight, and runnable via `RunWorkflowIntent` (model-free: it enqueues into `InboxStore`, which the app drains on activation).

11. UI

SwiftUI views bind directly to the orchestrator's published state:

- **ChatView** + **ChatViewComponents** — the conversational surface and the **voice-first input bar** (mic is the default action when the field is empty; becomes stop while thinking, send when there's text); also camera, photo, and image-creation entry points.
- **MessageBubble** — renders the five message kinds, images, and the streaming indicator.
- **AgentTraceView** — the glass-box reasoning timeline (route → memory → plan → steps → tool → critic → final), color-coded with latency pills, expandable per step.
- **Management surfaces:** `MemoryView`, `SettingsView`, `ConnectorsView` (+ subviews), `WorkflowsView`, `ModelManagerView`, `PrivacyView`, `PrivateCloudView`, `CameraCaptureView`, `SessionListView`, `ImageCreationSupport`.

12. Utilities (cross-cutting)

Utility	Role
KeychainStore	Secure storage for API keys, server endpoint, attestation keys; readable by extensions after first unlock.
PIISanitizer	Regex scrub (emails, IDs, cards, phones, addresses) → (text, redactionCount) . The single funnel before any cloud call.
SessionStore / ConversationStore	Multi-session on-disk layout (index.json + per-session conversation.json , 200-message cap), with legacy migration.
SessionContextTracker	Lets the KV-cached LiteRT session feed only the new prompt delta.
StreamParser	SSEParser + GeminiSSEParser parse streaming SSE responses.
StreamStopFilter	Stops generation at the first stop marker for engines that don't self-stop.
JSONSchema / JSONValue	Untyped JSON for tool parameters/arguments.
SpeechRecognizer / SpeechSynthesizer	On-device dictation (auto-send on final) and TTS (the talk-back half of the loop).
DeviceAttestation	App Attest identity for the private server (dev bearer token on Simulator/CI).
SpotlightIndexer	Indexes Workflow names (not prompts) into CoreSpotlight / Siri Suggestions.

13. End-to-end: one message, start to finish

Example: "What's the weather today, and then write me a reminder email about it."

1. **Entry** — ChatView (voice or text) calls orchestrator.run(userMessage:imageData:). The user message is appended; the privacy ledger begins a new turn.
2. **Classify** — TaskClassifier embeds it (.webInfo, high confidence); MemoryStore injects any relevant [M#] notes.
3. **Route** — ModelClassifier → .onDeviceAgent; the "weather"/"reminder" device-tool signal also forces the agent path.
4. **Gate** — "deep reasoning" on + PlanGate.shouldPlan() sees the "and then" sequencing → the plan pipeline runs.
5. **Plan** — Planner returns two steps: researcher → web_search, then writer → draft_email.
6. **Execute** — step 1 runs a headless ReAct loop with the researcher's tool subset (calls web_search after the capability/confirmation gate, ledgered); the result lands in the scratchpad. Step 2's writer drafts the email using that context.
7. **Synthesize** — the two results are combined into one answer.
8. **Critique** — Critic approves (or asks for one revision).
9. **Present** — the answer streams into the chat bubble; AgentTraceView shows every step; the privacy chip summarizes egress (here: one Brave search). If "Speak Responses" is on, SpeechSynthesizer reads it aloud.
10. **Persist** — the session is saved and floated to the top of the drawer.

14. Testing

~51 test files (the README cites ~130 individual tests): tool-call parsing, calculator edge cases (NaN/Inf/overflow/arity), the classifiers (with a `ClassifierEvalTests` accuracy harness — 86.7% task / 98.3% route, 70% floor), tokenizer, stream stop-filter, PII scrubbing, Keychain, memory store + retrieval, conversation/session persistence, planner/critic/plan-pipeline, MCP catalog/registry/proxy/auth migration, REST connectors, device attestation, the private-compute client, speech synthesis, Spotlight indexing, and the full orchestrator ReAct loop. UI tests drive the real chat flow against the DEBUG `ScriptedModel`; real-model smoke tests are opt-in via `REAL_MODEL_TEST=1`.

```
xcodebuild -workspace Anu.xcworkspace -scheme Anu \
  -destination 'platform=iOS Simulator,name=iPhone 17 Pro' test
```

15. Key invariants (quick reference)

- One `@MainActor` orchestrator owns all conversation state; UI and Siri share it.
- Engines are interchangeable behind `LocalLanguageModel`; selection is `LiteRT` → `Core ML` → `Unavailable` (or the user's explicit pick).
- ReAct ≤ 6 iterations; planner ≤ 6 steps; one critic pass; one revision; 30s tool timeout.
- Nothing leaves the device without going through `PIISanitizer` and being recorded in `PrivacyLedger`; external/remote tools are capability- and confirmation-gated.
- Secrets live in the Keychain; memory and on-device speech never leave the device.
- No silent fake model in production (`UnavailableModel`); the `ScriptedModel` is `#if DEBUG`.
- The Xcode project is generated from `generate_xcodeproj.py`; open the `.xcworkspace`.

16. The agentic system, in depth (with code)

Anu is *agentic*: it does not just generate text from a prompt. For each message it **decides** a route, **acts** by calling tools, **observes** the results, and **iterates** — optionally with a planner that decomposes the goal and a critic that reviews the answer. All of it runs on the device; the cloud is one tool among many. The excerpts below are the real control flow (lightly trimmed).

16.1 Routing every message — the entry point

Before any token is generated, two on-device classifiers decide which path the message takes, memory is injected, and image/fast-mode/device-tool overrides are applied. The route then dispatches to one of three execution paths (with the deep planner pipeline gated behind a setting *and* a structural check).

```
// AgentOrchestrator.run(userMessage:imageData:) – two on-device decisions
let classification = TaskClassifier.shared.classify(userMessage)
let image = imageData.flatMap(Self.cgImage(from:))

// NotebookLM-style memory: relevant notes injected as numbered [M#] sources
var memoryContext: String? = nil
if activeSession?.memoryScope != .isolated {
    (usedMemories, memoryContext) = MemoryStore.shared.context(for: userMessage)
}

let fastMode = UserDefaults.standard.bool(forKey: "fast_mode")
var route: ModelRoute = (image != nil || fastMode)
    ? .onDeviceChat
    : ModelClassifier.route(task: classification,
                           cloudAvailable: ModelClassifier.cloudAvailable)

// Clock/calendar/reminder/contact questions must run a tool to be correct
if route == .onDeviceChat, !fastMode, ModelClassifier.needsDeviceTool(userMessage) {
    route = .onDeviceAgent
}

switch route {
case .onDeviceChat:
    await chatTurn(image: image, memoryContext: memoryContext)
case .onDeviceAgent:
    let deepReasoning = UserDefaults.standard.bool(forKey: "deep_reasoning")
    if deepReasoning && PlanGate.shouldPlan(task: classification, message: userMessage) {
        await planPipeline(goal: userMessage, memoryContext: memoryContext) //
    plan→execute→critique
    } else {
        await reactLoop(startingIteration: 0, memoryContext: memoryContext) // default
    fast path
    }
case .cloudEscalate:
    await cloudTurn(userMessage: userMessage)
}
```

16.2 The ReAct loop — reason → act → observe

The default agent path is a bounded loop (≤ 6 iterations). Each turn builds a prompt that includes the tool catalog, streams the model, then parses the output into a structured decision: answer, call a tool, or escalate. Tool-call JSON is hidden from the chat and surfaced as a tool card plus a reasoning-trace step; the observation feeds the next iteration.

```
// AgentOrchestrator.reactLoop — bounded reason → act → observe (maxIterations = 6)
while iteration < maxIterations {
    iteration += 1

    let window = ConversationWindow.windowed(conversationHistory)
    let prompt = GemmaChatTemplate.format(messages: window,
                                          tools: toolRegistry.allTools,
                                          memoryContext: memoryContext)

    var rawResponse = ""
    let stream = try await model.generate(prompt: prompt, config: .agentFromSettings)
    for await token in stream {
        rawResponse += token
        await updateStreamingMessage(token: token, iteration: iteration)
    }

    // Parse the model's output into a structured decision
    switch ResponseParser.parse(rawResponse) {
    case .answerLocally(let answer): // done — keep the streamed bubble,
return
        finalizeStreamingMessage()
        conversationHistory.append(.assistant(answer))
        return
    case .callTool(let info): // run a tool, observe, loop again
        discardStreamingMessage() // hide raw JSON from chat
        await performToolCall(info, status: .callingTool(info.name))
    case .escalateToGemini(let task, let context): // hand off to the cloud
        discardStreamingMessage()
        await performToolCall(ToolCallInfo(name: "escalate_to_gemini", ...), status:
        .escalating)
    }
}
```

16.3 Planner → specialists → critic — the deep pipeline

For clearly multi-step requests (and only when "deep reasoning" is on), the orchestrator runs a hierarchical pipeline. The **Planner** emits a JSON plan in one deterministic call, assigning each step to a **specialist**; each specialist runs the same ReAct loop but with a focused prompt and a restricted tool subset; results are synthesized, then a **Critic** reviews the draft and may request exactly one revision.

```
// Planner.systemPrompt – decompose the goal into the FEWEST ordered steps
"""
You are the Planner. [[PLANNER]]
Break the user's request into the SMALLEST number of ordered steps ...
Available specialists: researcher, coder, mathematician, writer, generalist.
Rules:
- Use as FEW steps as possible. A simple request is ONE step.
- Each step must be a concrete instruction a single specialist can do.
- Later steps may rely on earlier steps' results.

Respond with ONLY this JSON, nothing else:
{"plan": [
  {"step": 1, "description": "<what to do>", "specialist": "<name>", "tool_hint": "
<optional>"}
]}
"""
```

```
// PlanModels.Specialist – each persona runs ReAct with only the tools it needs
var allowedToolNames: Set<String> {
  switch self {
    case .researcher:    return ["web_search", "datetime", "calendar", "reminders",
"contacts"]
    case .coder:         return ["calculator", "escalate_to_gemini"]
    case .mathematician: return ["calculator", "unit_converter"]
    case .writer:        return ["escalate_to_gemini", "datetime"]
    case .generalist:    return [/* all registered tools */]
  }
}
```

```
// Critic.parse – reflection: approve, or ask for exactly ONE revision.
// Bounded by design; any parse ambiguity defaults to .approved (never blocks an answer).
if parsed.verdict.lowercased().hasPrefix("approve") { return .approved }
if let instruction, !instruction.isEmpty { return .revise(instruction: instruction) }
return .approved
```

17. On-device execution, in depth (with code)

The on-device story is not one model call — it is a chain of decisions and engines that keep work local by default and make the cost of leaving the device visible. This section shows the pieces that make that real.

17.1 Routing without a model round-trip

The first two decisions cost *no* model inference and no network. `TaskClassifier` embeds the message with Apple's bundled `NLEmbedding` and assigns it to the nearest task centroid; `ModelClassifier` turns that into a route, biased toward staying on-device.

```
// TaskClassifier – NLEmbedding sentence vectors; nearest task centroid.
// Centroids are precomputed once from a few seed utterances per task type.
private init() {
    embedding = NLEmbedding.sentenceEmbedding(for: .english)
    for (type, examples) in Self.seeds {
        let vectors = examples.compactMap { embedding?.vector(for: $0.lowercased()) }
        centroids[type] = mean(vectors) // centroid = mean of seed vectors
    }
}

func classify(_ text: String) -> TaskClassification {
    guard let vector = embedding?.vector(for: text.lowercased()) else {
        return Self.keywordFallback(text) // unsupported locale → keyword rules
    }
    var best: (TaskType, Double) = (.generalQA, -1)
    for (type, centroid) in centroids {
        let score = Self.cosine(vector, centroid) // cosine similarity
        if score > best.1 { best = (type, score) }
    }
    return TaskClassification(type: best.0, confidence: best.1)
}
```

```
// ModelClassifier – privacy & latency first; stay local unless cloud clearly wins.
static let confidenceFloor = 0.45

static func route(task: TaskClassification, cloudAvailable: Bool) -> ModelRoute {
    guard task.confidence >= confidenceFloor else { return .onDeviceAgent } // unsure → agent
    switch task.type {
        case .casualChat, .generalQA: return .onDeviceChat // fast, no tools
        case .math, .webInfo: return .onDeviceAgent // needs tools, still local
        case .codeGen, .longWriting: return cloudAvailable ? .cloudEscalate : .onDeviceChat
    }
}
```

17.2 The swappable brain

Every engine conforms to one `LocalLanguageModel` protocol, so the orchestrator, tools, and UI never know which backend is running. `ModelFactory` honors the user's explicit choice and otherwise falls back to a priority chain — and a missing model is an explicit error, never a silent fake.

```
// ModelFactory – honor the user's choice; default "automatic" priority chain.
static func makeModel(...) -> any LocalLanguageModel {
    #if DEBUG
        if UserDefaults.standard.bool(forKey: "scripted_model") { return ScriptedModel() }
    #endif
    switch selection {
    case .appleFoundation: return makeAppleModel()
    case .liteRT:           return makeLiteRTModel() ?? UnavailableModel(...)
    case .coreML:           return makeCoreMLModel() ?? UnavailableModel(...)
    case .privateCloud:     return makePrivateCloudModel()
    case .automatic:        return makeAutomatic()
    }
}

// LiteRT (KV-cached) → Core ML (fixed-shape) → UnavailableModel
private static func makeAutomatic(config: ModelConfig) -> any LocalLanguageModel {
    if let lite = makeLiteRTModel() { return lite }
    if let core = makeCoreMLModel(config: config) { return core }
    return UnavailableModel() // explicit error – no silent fake
}
```

17.3 Fast multi-turn inference — LiteRT KV-cache reuse

The primary engine keeps a live MediaPipe session whose KV cache persists across turns. When the next prompt simply extends what the session has already consumed, Anu feeds only the new delta and skips re-priming the whole history — the main reason multi-turn chat stays fast on a 4B-class model.

```
// LiteRTGemmaModel.generate – reuse the session's KV cache across turns.
let feed = tracker.feed(for: prompt)
if image == nil, let existing = session,
    sessionTemperature == config.temperature,
    case .append(let delta) = feed {
    try existing.addQueryChunk(inputText: delta) // cheap: skip the prefill
    activeSession = existing
} else {
    let opts = LlmInference.Session.Options()
    opts.temperature = config.temperature
    opts.topP = config.topP; opts.topK = config.topK
    opts.enableVisionModality = (image != nil) // vision set at session creation
    activeSession = try LlmInference.Session(llmInference: llm, options: opts)
    try activeSession.addQueryChunk(inputText: prompt)
    if let image { try activeSession.addImage(image: image) }
    session = activeSession
}
```

17.4 Nothing leaves silently

On-device generation never touches the network. When a task *does* escalate, a single funnel ([PIISanitizer](#)) scrubs personal data first and reports a redaction count, and every outward-facing tool is classified by its side effect so the capability/confirmation gates and the privacy ledger can act on it.

```
// PIISanitizer.sanitize – the single funnel before any cloud call.
// Returns scrubbed text AND a redaction count (surfaced in the privacy ledger).
count += replaceAll(pattern: #"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"#, with: "[EMAIL]")
count += replaceAll(pattern: #"^\b\d{3}-\d{2}-\d{4}\b"#, with: "[ID]")
count += replaceAll(pattern: #"^\b(?:\d[ -])?\{13,16\}\b"#, with: "[NUMBER]")
// Phone numbers + street addresses via NSDataDetector → [PHONE] / [ADDRESS]
return Result(text: output, redactions: count)
```

```
// Tool – every capability the model can invoke; the side effect drives gating.
protocol Tool: Sendable {
    var name: String { get }
    var description: String { get }
    var parameters: JSONSchema? { get }
    var sideEffect: ToolSideEffect { get } // .readOnly | .external(capability:)
    var requiresConfirmation: Bool { get } // outward actions confirm before each run
    func execute(arguments: JSONValue) async throws -> String
}

enum ToolSideEffect { case readOnly; case external(capability: ConnectorCapability) }
```